Data Ingestion – Event-Driven & Streaming

1. Identitas Modul

Nama mata kuliah	:	Praktikum Data Processing Workflow
Topik	:	Data Ingestion – Event-Driven & Streaming
Durasi	:	100 Menit
Sub-CPMK	:	Mahasiswa mampu merancang dan mengeksekusi alur data asynchronous menggunakan arsitektur event-driven untuk kebutuhan streaming.
Indikator Keberhasilan	:	1. Menentukan konfigurasi Consumer Groups, Partitions, dan Offsets yang tepat. 2. Menganalisis perbedaan hasil pengiriman pesan At-least-once vs Exactly-once. 3. Mengoperasikan message broker via Docker Compose serta menjalankan skrip producer dan consumer secara asinkron.
Tools	:	Docker Compose, Apache Kafka / RabbitMQ, Python

2. Teori Esensial

On progress

3. Eksperimen & Analisis

Artefak praktikum yang disediakan :

```
dpw-modul-03/
│
├─── docker-compose.yml
├─── requirements.txt
│
├─── app/
│    ├─── producer.py
│    ├─── consumer.py
│    ├─── consume_transform_produce.py
│    └─── inspect.py
│
├─── data/
│    └─── events.json
│
├─── diagrams/
│    ├─── 01-event-driven-flow.puml
│    ├─── 02-topic-partition.puml
│    ├─── 03-consumer-group.puml
│    ├─── 04-offset-flow.puml
│    └─── 05-delivery-semantics.puml
│
└─── evidence/
     └─── experiment-report.md
```

Aturan perubahan

Selama praktikum:

- jangan mengubah docker-compose.yml, kecuali bagian yang **secara eksplisit diminta**;
- jangan mengubah source code Python;
- jangan mengganti broker;
- jangan membuat topic dengan nama berbeda dari instruksi;
- gunakan group.id yang diberikan pada setiap eksperimen;
- jangan menggunakan auto.offset.reset=latest ketika eksperimen membutuhkan pembacaan dari awal;
- jangan menjalankan consumer menggunakan group yang sama secara tidak sengaja antar-eksperimen;
- setiap eksperimen harus menggunakan evidence yang berbeda;
- setelah eksperimen selesai, kembalikan environment ke kondisi yang dinyatakan dalam instruksi berikutnya.

> **Catatan:** Anda tidak perlu menghafalkan seluruh command Kafka. Fokus pada hubungan antara tindakan yang dilakukan, perubahan konfigurasi, perilaku sistem, dan bukti yang dihasilkan.

Broker yang digunakan

Praktikum menggunakan **Apache Kafka** sebagai satu-satunya message broker.
Tidak diperlukan implementasi RabbitMQ.

Konfigurasi dasar

Kafka dijalankan sebagai single broker menggunakan Docker Compose.
Topic utama yang digunakan selama praktikum:

```
dpw3-events
```

Topic tersebut memiliki:

```
3 partitions
```

3.1 Verifikasi Environment

Waktu: ±5 menit

Tujuan

Memastikan environment praktikum siap digunakan sebelum menjalankan Kafka.

Prerequisite

Pastikan terminal berada pada folder:

```
dpw-modul-03/
```

Langkah 1 — Verifikasi Docker

Jalankan:

```
docker --version
```

Kemudian:

```
docker compose version
```

Expected result

> Kedua command menampilkan versi Docker dan Docker Compose tanpa error.

Langkah 2 — Verifikasi Python

Jalankan:

```
python --version
```

Kemudian:

```
python -c "import confluent_kafka; print(confluent_kafka.version())"
```

Jika library belum tersedia, jalankan:

```
pip install -r requirements.txt
```

Kemudian ulangi command sebelumnya.

Expected result

> Python dapat dijalankan dan library Kafka Python berhasil di-import.

Jika gagal

Jangan mengubah source code.
Gunakan pesan error untuk menentukan apakah masalah berada pada:

- Docker;
- Docker Compose;
- Python;
- Python package.

Verification

Seluruh command berikut harus berhasil:

```
docker --version
docker compose version
python --version
python -c "import confluent_kafka; print(confluent_kafka.version())"
```

Evidence

Simpan screenshot:

E01-environment.png

Checkpoint

> Lanjutkan jika Docker, Docker Compose, Python, dan Kafka Python client dapat digunakan tanpa error.

3.2 Menjalankan Message Broker

Waktu: ±5 menit

Tujuan

Menjalankan Apache Kafka sebagai message broker menggunakan Docker Compose.

Prerequisite

Bagian 3.1 berhasil.

Step 1 — Build

Periksa konfigurasi:

```
docker compose config
```

Command harus selesai tanpa error.

Step 2 — Run

Jalankan Kafka:

```
docker compose up -d
```

Periksa container:

```
docker compose ps
```

Expected result

> Container Kafka berada dalam kondisi berjalan.

Step 3 — Buat Topic

Buat topic utama:

```
docker compose exec broker \
/opt/kafka/bin/kafka-topics.sh \
--create \
--topic dpw3-events \
--bootstrap-server localhost:9092 \
--partitions 3 \
--replication-factor 1
```

Periksa konfigurasi topic:

```
docker compose exec broker \
/opt/kafka/bin/kafka-topics.sh \
--describe \
--topic dpw3-events \
--bootstrap-server localhost:9092
```

Expected result

Informasi topic menunjukkan:

```
PartitionCount: 3
ReplicationFactor: 1
```

Observe

Catat bahwa satu topic:

```
dpw3-events
```

memiliki tiga partition.

Jika gagal

Periksa status container:

```
docker compose ps
```

Kemudian:

```
docker compose logs broker
```

Identifikasi apakah masalah berasal dari container atau Kafka.

Fix

Jika broker belum berjalan:

```
docker compose down
```

Kemudian:

```
docker compose up -d
```

Ulangi pemeriksaan.

Verification

Jalankan kembali:

```
docker compose exec broker \
/opt/kafka/bin/kafka-topics.sh \
--describe \
--topic dpw3-events \
--bootstrap-server localhost:9092
```

Evidence

Simpan:

```
E02-broker.png
```

Screenshot harus menunjukkan broker aktif dan topic dpw3-events dengan tiga partition.

Checkpoint

> Lanjutkan jika Kafka berjalan dan topic dpw3-events memiliki tiga partition.

3.3 Producer–Consumer Asynchronous

Waktu: ±10 menit

Tujuan

Menjalankan producer dan consumer sebagai proses terpisah dan mengamati aliran event melalui Kafka.

Prerequisite

Kafka broker dan topic dpw3-events telah tersedia.

Step 1 — Run Consumer

Buka terminal pertama.

Jalankan:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-basic \
  --reset earliest
```

Biarkan consumer tetap berjalan.

Step 2 — Run Producer

Buka terminal kedua.

Jalankan:

```
python app/producer.py \
  --topic dpw3-events \
  --count 10
```

Expected result

Producer mengirim event:

```
event-001
event-002
...
event-010
```

Consumer menerima event tersebut.

Perhatikan informasi:

```
event_id
partition
offset
```

Contoh:

```
event-001  partition=0  offset=0
event-002  partition=1  offset=0
event-003  partition=2  offset=0
```

Nilai partition dan offset dapat berbeda dari contoh.

Observe

Catat minimal tiga event yang menunjukkan:

- event_id;
- partition;
- offset.

Pertanyaan analisis

> Apakah producer harus menunggu consumer selesai memproses event sebelum producer dapat mengirim event berikutnya?

Jawab maksimal **2–3 kalimat**.

Failure Scenario

Hentikan consumer menggunakan:

CTRL + C

Kemudian jalankan producer kembali:

```
python app/producer.py \
```

```
  --topic dpw3-events \
  --count 5
```

Perhatikan apakah producer masih dapat mengirim event.

Kemudian jalankan consumer kembali:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-basic \
  --reset earliest
```

Diagnosis

Jika producer tetap dapat mengirim ketika consumer tidak aktif, hal tersebut menunjukkan bahwa producer dan consumer tidak harus berjalan sebagai satu proses yang sama.

Kafka menjadi perantara penyimpanan dan distribusi event.

Verification

Consumer kembali dapat membaca event yang telah dikirim.

Evidence

Simpan:

```
E03-producer-consumer.png
```

Checkpoint

> Anda dapat menjelaskan mengapa producer dan consumer dapat berjalan sebagai proses yang terpisah.

3.4 Eksperimen Topic & Partition

Waktu: ±10 menit

Tujuan

Mengamati bagaimana event didistribusikan ke beberapa partition dalam satu topic.

Prerequisite

Topic dpw3-events memiliki tiga partition.

Step 1 — Produce

Jalankan:

```
python app/producer.py \
  --topic dpw3-events \
  --count 15 \
  --key-mode entity
```

Step 2 — Consume

Jalankan:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-partition \
  --reset earliest
```

Observe

Perhatikan nilai:

```
partition=0
partition=1
```

```
partition=2
```

Tidak semua event harus berada pada partition yang sama.

Buat catatan sederhana:

event_id	partition	offset
event-001	...	...
event-002	...	...
event-003	...	...
event-004	...	...
event-005	...	...

Pertanyaan analisis

Pertanyaan 1

Mengapa event dapat berada pada partition yang berbeda?

Pertanyaan 2

Apakah offset 5 pada partition 0 sama dengan offset 5 pada partition 1?

Jawab maksimal **2–3 kalimat per pertanyaan**.

Diagnosis

Perhatikan bahwa offset tidak berlaku secara global untuk seluruh topic.

Offset merupakan posisi record pada **partition tertentu**.

Verification

Jalankan:

```
docker compose exec broker \
/opt/kafka/bin/kafka-get-offsets.sh \
--bootstrap-server localhost:9092 \
--topic dpw3-events
```

Amati offset masing-masing partition.

Evidence

Simpan:

```
E04-partition.png
```

Checkpoint

> Anda dapat menunjukkan bahwa satu topic dapat memiliki beberapa partition dan bahwa offset diamati pada level partition.

3.5 Eksperimen Consumer Group

Waktu: ±10 menit

Tujuan

Mengamati bagaimana partition dibagikan kepada beberapa consumer dalam consumer group yang sama.

Prerequisite

Topic dpw3-events tersedia.

Step 1 — Consumer 1

Terminal pertama:

```
python app/consumer.py \
```

```
  --topic dpw3-events \
  --group dpw3-group \
  --consumer-name C1
```

Step 2 — Consumer 2

Terminal kedua:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-group \
  --consumer-name C2
```

Kedua consumer menggunakan:

group.id = dpw3-group

Step 3 — Produce

Terminal ketiga:

```
python app/producer.py \
  --topic dpw3-events \
  --count 20
```

Observe

Perhatikan partition yang diterima oleh:

```
C1
C2
```

Partition akan dibagikan kepada anggota consumer group.

Pembagian persis dapat berbeda.

Step 4 — Failure

Hentikan C2:

```
CTRL + C
```

Amati log C1.

Kafka akan melakukan **rebalance** dan partition yang sebelumnya ditangani C2 dapat diberikan kepada C1.

Diagnosis

Yang berubah bukan jumlah partition pada topic.

Yang berubah adalah:

partition assignment di dalam consumer group.

Verification

Gunakan:

```
docker compose exec broker \
/opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--describe \
--group dpw3-group
```

Perhatikan:

```
PARTITION
CURRENT-OFFSET
CONSUMER-ID
```

Evidence

Simpan:

```
E05-consumer-group.png
```

Evidence harus menunjukkan kondisi sebelum dan/atau sesudah consumer dihentikan.

Analisis teknis

1. Jika topic memiliki tiga partition dan consumer group memiliki dua consumer, partition dapat dibagi di antara dua consumer.
2. Jika satu consumer keluar, Kafka melakukan rebalance untuk mendistribusikan kembali partition kepada anggota group yang tersisa.

Checkpoint

> Anda dapat menjelaskan hubungan antara partition, consumer group, consumer, dan rebalance.

3.6 Eksperimen Offset

Waktu: ±10 menit

Tujuan

Mengamati fungsi committed offset sebagai posisi konsumsi consumer group.

Prerequisite

Gunakan consumer group baru:

```
dpw3-offset
```

Step 1 — Consume

Jalankan:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-offset \
  --reset earliest \
  --stop-after 5
```

Consumer berhenti setelah memproses lima event.

Step 2 — Inspect Offset

Jalankan:

```
docker compose exec broker \
/opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--describe \
--group dpw3-offset
```

Perhatikan:

```
CURRENT-OFFSET
LOG-END-OFFSET
LAG
```

Step 3 — Run Again

Jalankan kembali:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-offset
```

Observe

Consumer melanjutkan konsumsi berdasarkan posisi yang telah disimpan oleh consumer group.

Pertanyaan analisis

> Mengapa consumer tidak kembali membaca seluruh event dari awal?
> Jawab maksimal 3 kalimat.

Diagnosis

Consumer group menggunakan committed offset untuk menentukan posisi konsumsi berikutnya.

Verification

Periksa kembali:

docker compose exec broker \

```
/opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--describe \
--group dpw3-offset
```

Amati perubahan CURRENT-OFFSET dan LAG.

Evidence

Simpan:

```
E06-offset.png
```

Checkpoint

> Anda dapat menjelaskan bahwa offset digunakan consumer group untuk mempertahankan posisi konsumsi.

3.7 Failure Injection

Waktu: ±10 menit

Tujuan

Membuktikan bahwa kegagalan consumer dapat menyebabkan perubahan assignment dan consumer group dapat melanjutkan konsumsi.

Prerequisite

Gunakan:

```
topic  : dpw3-events
group  : dpw3-failure
```

Step 1 — Consumer 1

Terminal pertama:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-failure \
  --consumer-name C1 \
  --fail-after 3
```

Consumer akan berhenti setelah sejumlah event sesuai parameter failure.

Step 2 — Consumer 2

Terminal kedua:

```
python app/consumer.py \
  --topic dpw3-events \
```

```
  --group dpw3-failure \
  --consumer-name C2
```

Step 3 — Produce

Terminal ketiga:

```
python app/producer.py \
  --topic dpw3-events \
  --count 10
```

Step 4 — Observe Failure

C1 akan mengalami failure.

Amati C2.

Perhatikan apakah terjadi perubahan assignment setelah C1 berhenti.

Diagnosis

Kegagalan C1 mengubah membership consumer group.

Kafka kemudian melakukan rebalance agar partition yang sebelumnya ditangani C1 dapat ditangani oleh member group yang masih aktif.

Fix

Tidak perlu memperbaiki source code.

Jalankan kembali consumer:

```
python app/consumer.py \
  --topic dpw3-events \
  --group dpw3-failure \
  --consumer-name C1-recovery
```

Verification

Periksa:

```
docker compose exec broker \
/opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--describe \
--group dpw3-failure
```

Pastikan consumer group kembali memiliki member aktif dan proses konsumsi dapat dilanjutkan.

Evidence Chain

Susun:

```
FAIL
  ↓
OBSERVE
  ↓
DIAGNOSE
  ↓
FIX
  ↓
VERIFY
```

Evidence

Simpan:

```
E07-failure.png
```

Checkpoint

> Anda dapat menunjukkan bukti bahwa:
> 1. consumer mengalami failure;
> 2. consumer group melakukan recovery/rebalance;
> 3. consumer lain dapat mengambil alih partition.

3.8 At-least-once vs Exactly-once

Waktu: ±20 menit

Tujuan

Membuktikan perbedaan hasil **At-least-once** dan **Exactly-once** melalui failure yang terjadi pada saat pemrosesan.

Eksperimen menggunakan dataset kecil sehingga hasil dapat dihitung dan dibandingkan.

A. Eksperimen At-least-once

Prerequisite

Gunakan dataset yang telah disediakan dan mode:

```
atleast-once
```

Step 1 — Run

Jalankan:

```
python app/consume_transform_produce.py \
  --mode atleast-once \
  --fail-after-produce
```

Program sengaja berhenti setelah output dihasilkan tetapi sebelum offset berhasil di-commit.

Observe

Kondisi yang terjadi:

```
consume
   ↓
produce output
   ↓
FAIL
```

Output telah dihasilkan, tetapi posisi konsumsi belum tersimpan.

Step 2 — Restart

Jalankan:

```
python app/consume_transform_produce.py \
  --mode atleast-once
```

Observe

Message yang belum committed dapat diproses kembali.

Periksa hasil output menggunakan:

```
python app/inspect.py
```

Catat:

```
Total output
Unique event_id
```

```
Duplicate event_id
```

Expected observation

Jumlah unique event_id tetap merepresentasikan event yang diproses, tetapi jumlah output dapat lebih besar karena terjadi pemrosesan ulang.

Contoh:

```
Total output   : 13
Unique event   : 10
Duplicate      : 3
```

Nilai aktual bergantung pada titik failure.

Diagnosis

Failure terjadi setelah output dihasilkan tetapi sebelum offset di-commit.

Akibatnya consumer group belum mengetahui bahwa event tersebut telah selesai diproses.

Ketika consumer dijalankan kembali, event dapat dibaca lagi.

Verification

Buktikan bahwa:

```
Total output > Unique event_id
```

B. Eksperimen Exactly-once

Step 1 — Run

Jalankan mode exactly-once:

```
python app/consume_transform_produce.py \
  --mode exactly-once \
  --fail-before-commit
```

Program sengaja mengalami failure sebelum transaction berhasil di-commit.

Observe

Secara konseptual:

```
consume
   ↓
transaction
   ↓
produce output
   ↓
FAIL sebelum commit
```

Output yang belum menjadi committed transaction tidak dianggap sebagai hasil akhir.

Step 2 — Restart

Jalankan kembali:

```
python app/consume_transform_produce.py \
  --mode exactly-once
```

Step 3 — Read Committed Output

Gunakan:

```
python app/inspect.py \
  --isolation read_committed
```

Expected observation

Hasil committed menunjukkan satu hasil untuk setiap event_id.

Contoh:

```
Total output   : 10
Unique event   : 10
Duplicate      : 0
```

Diagnosis

- Pada mode exactly-once, output record dan posisi konsumsi diperlakukan sebagai bagian dari transaction Kafka.
- Jika transaction gagal sebelum commit, hasil transaction tersebut tidak menjadi committed output.
- Ketika message diproses kembali, hanya hasil transaction yang berhasil di-commit yang terlihat oleh consumer read_committed.

Verification

Catat:

```
Total output
Unique event_id
Duplicate event_id
```

Kemudian bandingkan dengan eksperimen At-least-once.

Perbandingan Hasil

Isi tabel berikut berdasarkan hasil eksperimen:

Parameter	At-least-once	Exactly-once
Input event	10	10
Total output	...	...
Unique event	...	...
Duplicate output	...	...
Failure terjadi	Ya	Ya
Output transaction committed	...	...

Gunakan **hasil aktual eksperimen**, bukan angka contoh.

Pertanyaan Analisis

1. Mengapa At-least-once dapat menghasilkan duplicate output setelah failure?
2. Mengapa Exactly-once dapat menghindari duplicate **committed output** pada eksperimen ini?
3. Apa perbedaan hasil yang Anda amati antara kedua mode tersebut?

Batas jawaban: maksimal 3 kalimat per pertanyaan.

Catatan Penting

Exactly-once pada eksperimen ini dibuktikan dalam **alur Kafka**, yaitu hubungan antara input offset dan output Kafka transaction.

Hasil ini tidak berarti seluruh sistem eksternal secara otomatis menjadi exactly-once.

Evidence

Simpan:

```
E08-atleast-once.png
E09-exactly-once.png
```

Kedua evidence harus memperlihatkan hasil yang dapat dibandingkan.

Checkpoint

> Eksperimen dianggap berhasil jika Anda dapat menunjukkan berdasarkan hasil eksekusi bahwa:
>
> - At-least-once dapat menghasilkan duplicate output setelah failure;
> - Exactly-once menghasilkan satu committed output untuk setiap event pada eksperimen;
> - perbedaan tersebut dapat dijelaskan berdasarkan posisi commit dan transaction.

3.9 Evidence Chain

Waktu: ±5 menit

Anda tidak perlu membuat laporan panjang.

Susun enam kelompok evidence berikut:

```
E01 — Environment
E02 — Broker
E03 — Producer & Consumer
E04 — Partition
E05 — Consumer Group & Offset
E06 — Failure & Delivery Semantics
```

Untuk setiap evidence, tuliskan:

```
Action:
Apa yang dilakukan?

Observation:
Apa yang terlihat?

Conclusion:
Apa yang dibuktikan?
```

Contoh

```
E04 — Partition

Action:
Mengirim 15 event ke topic dpw3-events.

Observation:
Event diterima pada partition 0, 1, dan 2.

Conclusion:
Topic dpw3-events menggunakan tiga partition
dan event dapat didistribusikan ke partition tersebut.
```

Untuk failure

Gunakan:

```
Action
   ↓
Observation
```

```
   ↓
Diagnosis
   ↓
Fix
   ↓
Verification
```

> **Ingat:** screenshot adalah bukti, bukan analisis.

3.10 Checkpoint

Waktu: ±5 menit

Jawab pertanyaan berikut tanpa melihat kembali langkah praktikum.

1. Apa fungsi partition pada Kafka?
2. Mengapa beberapa consumer dalam consumer group yang sama dapat menerima partition yang berbeda?
3. Apa fungsi committed offset?
4. Apa yang terjadi pada partition assignment ketika salah satu consumer dalam group mengalami failure?
5. Berdasarkan hasil eksperimen, apa perbedaan At-least-once dan Exactly-once?

Artefak yang Dikumpulkan

Kumpulkan hanya artefak yang diminta.

```
submission/
├── evidence/
│   ├── E01-environment.png
│   ├── E02-broker.png
│   ├── E03-producer-consumer.png
│   ├── E04-partition.png
│   ├── E05-consumer-group.png
│   ├── E06-offset.png
│   ├── E07-failure.png
│   ├── E08-atleast-once.png
│   └── E09-exactly-once.png
│
├── diagrams/
│   └── *.puml
│
└── experiment-report.md
```

Isi experiment-report.md

Gunakan struktur:

1. Partition Analysis
2. Consumer Group Analysis
3. Offset Analysis
4. Failure Analysis
5. At-least-once vs Exactly-once
6. Technical Reflection

Tidak perlu menyalin teori dari Bagian 2.

Fokus pada:

> **apa yang dilakukan, apa yang terjadi, mengapa terjadi, dan apa yang membuktikannya.**

Definition of Done

Praktikum dianggap selesai jika seluruh kondisi berikut terpenuhi:

- ☐ Environment berhasil diverifikasi.
- ☐ Kafka broker berhasil dijalankan.
- ☐ Topic dpw3-events berhasil dibuat.
- ☐ Topic memiliki tiga partition.
- ☐ Producer berhasil mengirim event.
- ☐ Consumer berhasil menerima event.
- ☐ Partition dan offset berhasil diamati.
- ☐ Consumer group dengan dua consumer berhasil dijalankan.
- ☐ Rebalance akibat consumer berhenti berhasil diamati.
- ☐ Committed offset berhasil diamati.
- ☐ Failure injection menghasilkan kondisi yang dapat diamati.
- ☐ Consumer group berhasil melakukan recovery.
- ☐ At-least-once menghasilkan duplicate output pada skenario failure.
- ☐ Exactly-once menghasilkan committed output tanpa duplicate pada eksperimen.
- ☐ Perbedaan At-least-once dan Exactly-once dapat dijelaskan berdasarkan evidence.
- ☐ Evidence Chain lengkap.
- ☐ Checkpoint dapat dijawab.

> **Selesai bukan berarti semua command berhasil. Selesai berarti Anda mampu menjalankan alur asynchronous, mengamati hubungan Partition–Consumer Group–Offset, membuat consumer mengalami failure, mengamati recovery, dan membuktikan perbedaan At-least-once dan Exactly-once berdasarkan hasil eksekusi.**

4. TANTANGAN MANDIRI

4.1 Skenario

Sebuah sistem **Payment Event Stream** menerima event pembayaran secara asynchronous melalui Apache Kafka.

Setiap event memiliki:

```
event_id | order_id | event_type
```

Contoh:

```
PAY-001 | ORD-01 | CREATED
PAY-002 | ORD-01 | PAID
PAY-003 | ORD-02 | CREATED
PAY-004 | ORD-01 | SHIPPED
```

Sistem memiliki requirement:

> **Event untuk order yang sama harus diproses secara berurutan, sedangkan event dari order yang berbeda dapat diproses secara paralel.**

Pada saat processing berlangsung, salah satu consumer dapat mengalami failure.

Starter project telah disediakan. Gunakan artefak tersebut sebagai dasar implementasi.

Batasan:

- Gunakan Docker Compose, Apache Kafka, dan Python.
- Gunakan topic dan dataset yang telah disediakan.
- Tidak perlu menggunakan teknologi tambahan.
- Tidak perlu membuat sistem dari awal.

4.2 Analisis Arsitektur

Tentukan konfigurasi yang paling sesuai untuk kasus:

1. jumlah partition;
2. consumer group;
3. jumlah consumer;
4. event key untuk menjaga ordering;
5. strategi offset/commit;
6. delivery semantics yang dapat dipertanggungjawabkan.

Tuliskan keputusan dan alasannya secara singkat pada:

```
technical-analysis.md
```

Maksimal 150 kata.

> Tidak ada satu konfigurasi yang harus sama. Keputusan harus konsisten dengan requirement dan dapat dibuktikan melalui hasil eksekusi.

4.3 Desain Event Flow

Gunakan:

```
diagrams/starter-event-flow.puml
```

Buat:

```
architecture.puml
architecture.png
```

Diagram minimal menunjukkan:

```
Producer
   ↓
Topic
   ↓
Partition
   ↓
Consumer Group
   ↓
Consumer
   ↓
Processing
   ↓
Offset
```

Tunjukkan bagaimana event dari **order yang sama** dipetakan agar ordering dapat dipertahankan.

4.4 Failure Scenario

Jalankan producer dan consumer berdasarkan konfigurasi yang telah dipilih.
Kemudian **hentikan salah satu consumer saat processing berlangsung**.
Jalankan kembali consumer menggunakan consumer group yang sama.
Amati:

- perubahan partition assignment;
- offset;
- event yang diproses kembali;
- kemungkinan duplicate processing;
- keberlanjutan processing setelah recovery.

Tuliskan secara singkat:

```
Action:
...

Observation:
...

Diagnosis:
...

Fix:
...

Verification:
...
```

Maksimal 5 kalimat.

4.5 Pembuktian

Kumpulkan tiga evidence berikut.

E01 — Configuration

Buktikan:

```
partition
consumer group
consumer
event key
offset
```

```
E01-configuration.png
```

E02 — Ordering

Buktikan bahwa event untuk satu order diproses sesuai urutan.
E02-ordering.png

E03 — Failure & Recovery

Buktikan:

```
Failure
```

```
   ↓
Recovery
   ↓
Processing Continues
```

Sertakan bukti yang mendukung delivery behavior yang Anda pilih.

```
E03-failure-recovery.png
```

Evidence Chain

Gunakan pola:

```
Action
  ↓
Observation
  ↓
Diagnosis
  ↓
Fix
  ↓
Verification
```

Submission

submission/

```
├─── architecture.puml
├─── architecture.png
├─── technical-analysis.md
├─── changed-files/
└─── evidence/
    ├─── E01-configuration.png
    ├─── E02-ordering.png
    └─── E03-failure-recovery.png
```

Definition of Done

- ☐ Architecture decision dibuat.
- ☐ Partition, consumer group, consumer, key, dan offset ditentukan.
- ☐ Producer dan consumer berjalan asynchronous.
- ☐ Ordering terbukti.
- ☐ Consumer failure berhasil dilakukan.
- ☐ Recovery berhasil.
- ☐ Delivery behavior dapat dijelaskan berdasarkan evidence.
- ☐ Architecture diagram dan tiga evidence tersedia.

> **Jangan mengulang eksperimen Bagian 3. Gunakan hasil eksperimen tersebut untuk mengambil keputusan engineering dan membuktikannya.**

5. EVALUASI AKHIR

Bagian ini merupakan evaluasi akhir berdasarkan artefak dan evidence yang telah dihasilkan pada Bagian 3 dan Bagian 4.

5.1 Evidence Submission

Tidak perlu membuat evidence baru. Gunakan artefak dan evidence dari Bagian 3 dan Bagian 4.

5.2 Technical Evaluation

Evaluasi didasarkan pada:

- ketepatan konfigurasi partition, consumer group, consumer, key, dan offset;
- ketepatan architecture decision;
- pembuktian ordering;
- failure handling dan recovery;
- delivery semantics;
- kualitas evidence dan technical reasoning.

5.3 Refleksi Singkat

> **"Keputusan arsitektur apa yang paling menentukan perilaku pipeline saat consumer mengalami failure, dan apa konsekuensinya terhadap ordering, recovery, atau duplicate processing?"**

Jawab dalam satu paragraf singkat berdasarkan hasil eksperimen dan evidence yang telah diperoleh.